

CO-1 ASSESSMENT TOOL-1

Q1) Trace of Lexical and Syntax Analysis

Consider the C statement:

```
result = (a + b) * c - 10;
```

A compiler processes this statement through several phases. The main phases involved here are **lexical analysis, syntax analysis, and semantic analysis**.

1. Lexical Analysis

Lexical analysis is the first phase of a compiler. The lexical analyzer, also called a **lexer**, reads the source program character by character and groups characters into meaningful units called **lexemes**.

Each lexeme is converted into a **token**.

For the statement:

```
result = (a + b) * c - 10;
```

Lexeme Token

result	Identifier (ID)
=	Assignment operator (ASSIGN)
(	Left parenthesis (LPAREN)
a	Identifier (ID)
+	Addition operator (PLUS)
b	Identifier (ID)
)	Right parenthesis (RPAREN)
*	Multiplication operator (MUL)
c	Identifier (ID)
-	Subtraction operator (MINUS)
10	Integer constant (NUM)
;	Semicolon (SEMICOLON)

Therefore, the sequence of tokens generated by the lexical analyzer is:

ID ASSIGN LPAREN ID PLUS ID RPAREN MUL ID MINUS NUM SEMICOLON

<ID,result>

<ASSIGN,=>

<LPAREN,(>

<ID,a>

<PLUS,+>

<ID,b>

<RPAREN,)>

<MUL,*>

<ID,c>

<MINUS,->

<NUM,10>

<SEMICOLON,;>

2. Operators, Operands and Delimiters

Operators:

- $= \rightarrow$ Assignment operator
- $+$ $\rightarrow$ Addition
- $*$ $\rightarrow$ Multiplication
- $-$ $\rightarrow$ Subtraction

Operands:

- result
- a
- b
- c
- 10

Delimiters / Punctuation:

- (→ Opening parenthesis
-) → Closing parenthesis
- ; → Statement terminator

3. Syntax Analysis

The **syntax analyzer**, or **parser**, receives the tokens produced by the lexical analyzer.

Its job is to check whether the tokens follow the grammatical rules of the C language and construct a **parse tree**, **syntax tree**, or **abstract syntax tree (AST)**.

A simplified grammar could be:

statement → ID = expression ;

expression → expression - term

| term

term → term * factor

| factor

factor → (expression)

| ID

| NUM

The parser processes:

result = (a + b) * c - 10;

According to the grammar, it identifies:

statement

|

+-- result

+-- =

+-- expression

+-- ;

The expression can be represented as:

-

/ \

```

      * 10
    /  \
   +    c
 /  \
a    b

```

Since the assignment is also part of the statement, the overall syntax structure can be represented as:

```

      =
    /  \
result -
      /  \
      * 10
    /  \
   +    c
 /  \
a    b

```

This is an **abstract syntax tree (AST)** representation.

4. Operator Precedence

The parser must correctly determine the order in which operations are performed.

In the expression:

$(a + b) * c - 10$

the parentheses have the highest priority.

Therefore:

$(a + b)$

is evaluated first.

Then:

$(a + b) * c$

is evaluated.

Finally:

$((a + b) * c) - 10$

is evaluated.

The relevant precedence is:

Priority Operator Meaning

Highest	()	Parentheses
Next	*	Multiplication
Next	+, -	Addition/Subtraction
Lower	=	Assignment

Thus, the expression is interpreted as:

$result = (((a + b) * c) - 10);$

The parentheses explicitly force $a + b$ to be evaluated before multiplication.

5. Operator Associativity

When operators have the same precedence, **associativity** determines their evaluation order.

The $+$ and $-$ operators have **left-to-right associativity**. Multiplication also has left-to-right associativity.

For example:

$a - b - c$

is interpreted as:

$(a - b) - c$

In our statement, after evaluating $(a + b) * c$, the subtraction is performed:

$((a + b) * c) - 10$

The assignment operator $=$ has a lower precedence than the arithmetic expression, so the complete right-hand side is determined before assigning its value to result.

6. Semantic Analysis

After syntax analysis, the compiler performs **semantic analysis**.

Semantic analysis checks whether the statement is **meaningful and semantically valid**, not just grammatically correct.

For this statement, the compiler checks things such as:

a. Declaration of variables

The compiler verifies that:

result

a

b

c

have been declared before they are used.

For example:

int a, b, c, result;

would make their use valid.

b. Type checking

The compiler checks that the operands have compatible data types.

For example, if:

a, b, c, result

are integers, then:

$(a + b) * c - 10$

is a valid integer expression.

c. Assignment compatibility

The compiler checks whether the resulting value of:

$(a + b) * c - 10$

can be assigned to:

result

without an invalid type conversion.

d. Identifier checking

The compiler checks that each identifier refers to a valid declared variable in the current scope.

7. Complete Compiler Trace

The complete flow can be summarized as:

Source Statement

↓

result = (a + b) * c - 10;



Lexical Analysis



Tokens



ID ASSIGN LPAREN ID PLUS ID RPAREN

MUL ID MINUS NUM SEMICOLON



Syntax Analysis



Syntax Tree / AST



=
/
result -
/
* 10
/
+ c
/
a b



Semantic Analysis



Check declarations, types, scope, and assignment compatibility

Q2) Analyze Lexical Ambiguity and Longest-Match Rule

Consider the C statement:

value = a >= b && c != d;

The **lexical analyzer** reads the statement from left to right and groups characters into **lexemes**. Each lexeme is converted into a corresponding **token**.

1. Lexemes and Tokens

The statement contains the following lexemes:

Lexeme	Token	Description
value	ID	Identifier
=	ASSIGN	Assignment operator
a	ID	Identifier
>=	REL_OP	Greater-than-or-equal operator
b	ID	Identifier
&&	LOGICAL_AND	Logical AND operator
c	ID	Identifier
!=	REL_OP	Not-equal operator
d	ID	Identifier
;	SEMICOLON	Statement terminator

Therefore, the token stream is:

ID ASSIGN ID REL_OP ID LOGICAL_AND ID REL_OP ID SEMICOLON

With token values:

<ID,value>

<ASSIGN,=>

<ID,a>

<REL_OP,>=>

<ID,b>

<LOGICAL_AND,&&>

<ID,c>

<REL_OP,!=>

<ID,d>

<SEMICOLON,;>

2. Recognition of Identifiers

Identifiers are names used for variables, functions, arrays, etc.

Examples in the statement are:

value

a

b

c

d

A suitable regular expression for a C-style identifier is:

$[A-Za-z_][A-Za-z0-9_]*$

This means:

- The first character must be a letter or `_`.
- The remaining characters can be letters, digits, or `_`.

For example:

value → ID

a → ID

student1 → ID

_total → ID

But:

1value

is not a valid identifier because it begins with a digit.

3. Ambiguity Between > and >=

Consider:

a >= b

There are two possible token patterns:

>

>=

The character sequence:

>=

could potentially be interpreted as:

> =

That would mean:

RELATIONAL_OPERATOR

ASSIGNMENT_OPERATOR

However, this is **not** the correct interpretation.

The lexical analyzer uses the **longest-match rule**, also called the **maximal-munch rule**.

Longest-match rule

The lexer chooses the **longest valid token** beginning at the current position.

When it encounters:

>=

it checks:

>

and:

>=

Both can begin with >, but >= is the longer valid token.

Therefore:

>= → REL_OP

instead of:

> → REL_OP

= → ASSIGN

Example

Input: >=

↓

Check >

↓
Check >=
↓
Choose longer match
↓
REL_OP

4. Ambiguity Between ! and !=

A similar situation occurs with:

c != d

The possible patterns are:

!

!=

If the lexer simply recognized ! immediately, it could incorrectly tokenize:

!=

as:

! =

But in C, != is a single **not-equal relational operator**.

The longest-match rule makes the lexer select:

!= → REL_OP

because != is longer than !.

Thus:

c != d

becomes:

ID REL_OP ID

rather than:

ID NOT ASSIGN ID

5. Logical Operator &&

The statement also contains:

&&

This represents the **logical AND** operator.

A suitable token specification is:

$\&\& \rightarrow \text{LOGICAL_AND}$

The lexer recognizes both & characters together as one token.

For example:

a && b

becomes:

ID LOGICAL_AND ID

6. Assignment Operator =

The single equal sign:

=

is the assignment operator.

Its token specification is:

$= \rightarrow \text{ASSIGN}$

Therefore:

value = a

becomes:

ID ASSIGN ID

It is important to distinguish:

=

from:

>=

!=

because the latter are multi-character operators.

7. Suitable Token Specifications

The lexical analyzer can use the following specifications.

A. Identifier

$\text{ID} \rightarrow [\text{A-Za-z_}][\text{A-Za-z0-9_}]^*$

B. Relational Operators

The relational operators relevant here are:

$\text{REL_OP} \rightarrow >= \mid > \mid <= \mid < \mid == \mid !=$

For this particular statement, the lexer needs:

$>=$

$!=$

C. Logical Operators

$\text{LOGICAL_OP} \rightarrow \&\& \mid \parallel$

For this statement:

$\&\& \rightarrow \text{LOGICAL_AND}$

D. Assignment Operator

$\text{ASSIGN} \rightarrow =$

E. Semicolon

$\text{SEMICOLON} \rightarrow ;$

8. Why the Order of Patterns Matters

Overlapping token patterns should generally be specified with the **longer patterns before the shorter patterns**.

For example:

$>=$

$>$

should be considered in that order.

Similarly:

$!=$

$!$

and:

$\&\&$

$\&$

The reason is that the longer operator should be recognized as one token whenever possible

A suitable ordering is:

1. >=
2. <=
3. ==
4. !=
5. &&
6. ||
7. >
8. <
9. !
10. &
11. |
12. =
13. Identifier

However, in a lexer that correctly implements the **longest-match rule**, the physical order is often less important for operators of different lengths because the lexer selects the longest successful match. Still, placing longer/specific patterns first is a useful lexer-design practice and prevents problems in implementations where rule priority breaks ties or where maximal matching is not automatically enforced.

9. Step-by-Step Lexical Analysis

Consider:

value = a >= b && c != d;

Step 1

The lexer reads:

value

It matches the identifier pattern.

value → ID

Step 2

It reads:

=

There is no longer assignment token beginning with = in this statement.

$\Rightarrow$ ASSIGN

Step 3

It reads:

a

$a \rightarrow$ ID

Step 4

It encounters:

>=

Both > and >= are possible prefixes.

The longest valid match is:

>=

Therefore:

$\Rightarrow$ REL_OP

Step 5

It reads:

b

$b \rightarrow$ ID

Step 6

It encounters:

&&

The lexer recognizes the two characters together:

$\&\& \rightarrow$ LOGICAL_AND

Step 7

It reads:

c

$c \rightarrow \text{ID}$

Step 8

It encounters:

!=

Both ! and != could begin at this position.

The longest valid match is:

!=

Therefore:

$!= \rightarrow \text{REL_OP}$

Step 9

It reads:

d

$d \rightarrow \text{ID}$

Step 10

Finally:

;

is recognized as:

SEMICOLON

10. Final Token Stream

The complete result is:

Source:

value = a >= b && c != d;

Lexical Analyzer

↓

<ID,value>

<ASSIGN,=>

<ID,a>

<REL_OP,>=>

<ID,b>

<LOGICAL_AND,&&>

<ID,c>

<REL_OP,!=>

<ID,d>

<SEMICOLON,;>

Or simply:

ID ASSIGN ID REL_OP ID LOGICAL_AND ID REL_OP ID SEMICOLON

11. Importance of the Longest-Match Rule

The longest-match rule prevents incorrect tokenization.

Input Possible shorter match Longest match Correct token

>=	>	>=	REL_OP
!=	!	!=	REL_OP
&&	&	&&	LOGICAL_AND
<=	<	<=	REL_OP
==	=	==	EQUAL_OP

Thus, the lexer always attempts to consume the **maximum number of characters that form a valid token**.

Conclusion

In the statement:

value = a >= b && c != d;

the lexical analyzer identifies **identifiers** (value, a, b, c, d), the **assignment operator** (=), **relational operators** (>=, !=), the **logical AND operator** (&&), and the **semicolon**.

The main lexical ambiguities occur because >= starts with > and != starts with !. The **longest-match rule** resolves these ambiguities by selecting >= instead of > and != instead of !.

Therefore, overlapping multi-character operators should be handled before their shorter prefixes, ensuring that the input is converted into the correct token stream.

Q3) Determine Token Stream and Handle Lexical Errors

Consider the program fragment:

```
int total = 250;
```

```
float average = total / 5.0;
```

```
result@ = average + 10;
```

The **lexical analyzer** reads the program from left to right, groups characters into lexemes, and produces tokens. It also detects characters that do not belong to any valid token pattern.

1. Line 1: `int total = 250;`

Lexemes and Tokens

Lexeme	Token	Classification
int	INT	Keyword
total	ID	Identifier
=	ASSIGN	Operator
250	INT_CONST	Constant
;	SEMICOLON	Separator

Token stream

<INT>

<ID,total>

<ASSIGN>

<INT_CONST,250>

<SEMICOLON>

Or simply:

INT ID ASSIGN INT_CONST SEMICOLON

The spaces are ignored by the lexical analyzer because they are **white-space characters**.

2. Line 2: float average = total / 5.0;

Lexemes and Tokens

Lexeme	Token	Classification
float	FLOAT	Keyword
average	ID	Identifier
=	ASSIGN	Operator
total	ID	Identifier
/	DIV	Operator
5.0	FLOAT_CONST	Constant
;	SEMICOLON	Separator

Token stream

<FLOAT>

<ID,average>

<ASSIGN>

<ID,total>

<DIV>

<FLOAT_CONST,5.0>

<SEMICOLON>

Or:

FLOAT ID ASSIGN ID DIV FLOAT_CONST SEMICOLON

Here, 5.0 is recognized as a **floating-point constant**, not as three separate tokens.

3. Line 3: result@ = average + 10;

This line contains the **lexical error**.

Let's scan it from left to right.

result

The lexer recognizes:

result → ID

@

The character:

@

is **not a valid character in a normal C identifier or as a standalone C operator/separator**.

Therefore, @ causes a **lexical error**.

Remaining characters

After reporting the error, the lexer can continue scanning:

= → ASSIGN

average → ID

+ → PLUS

10 → INT_CONST

; → SEMICOLON

4. Lexical Error

The invalid character is:

@

So:

result@ = average + 10;

^

contains a lexical error at @.

The problem is detected during **lexical analysis**, before syntax analysis.

The lexer cannot create a normal C token for @ in this context.

5. Token Stream Up to the Error

Before reaching @, the lexer successfully recognizes:

result

Therefore, the token stream immediately before the error is:

ID(result)

Then:

@

causes:

LEXICAL_ERROR('@')

So the sequence is:

ID(result) LEXICAL_ERROR(@)

6. Token Stream After the Error

A good lexical analyzer should **report the error and recover**, rather than stopping the entire compilation immediately.

After skipping the invalid character @, the lexer continues with:

= average + 10;

It produces:

ASSIGN

ID(average)

PLUS

INT_CONST(10)

SEMICOLON

Therefore, the complete token stream for line 3 after error recovery is:

ID(result)

LEXICAL_ERROR(@)

ASSIGN

ID(average)

PLUS

INT_CONST(10)

SEMICOLON

7. Complete Token Stream

Combining all three lines:

Line 1

INT ID ASSIGN INT_CONST SEMICOLON

Line 2

FLOAT ID ASSIGN ID DIV FLOAT_CONST SEMICOLON

Line 3

ID LEXICAL_ERROR(@) ASSIGN ID PLUS INT_CONST SEMICOLON

Thus, the complete stream is:

INT ID ASSIGN INT_CONST SEMICOLON

FLOAT ID ASSIGN ID DIV FLOAT_CONST SEMICOLON

ID LEXICAL_ERROR(@) ASSIGN ID PLUS INT_CONST SEMICOLON

8. Token Classification

The tokens in the complete program can be classified as follows:

Token	Example	Classification
INT	int	Keyword
FLOAT	float	Keyword
ID	total, average, result	Identifier
INT_CONST	250, 10	Constant
FLOAT_CONST	5.0	Constant
ASSIGN	=	Operator
DIV	/	Operator
PLUS	+	Operator
SEMICOLON	;	Separator
LEXICAL_ERROR	@	Invalid character

9. How Lexical Error Recovery Works

When the lexer encounters an invalid character, it should:

1. **Detect the invalid character.**
2. **Report the error**, such as:

Lexical error: invalid character '@'

3. **Skip the invalid character.**
4. Continue scanning from the next character.
5. Generate tokens for the remaining valid input.

For example:

result@ = average + 10;

↓

Invalid @

↓

Report lexical error

↓

Skip @

↓

Continue scanning

↓

= average + 10 ;

This allows the compiler to find **additional errors** instead of stopping at the first error.

10. Error Recovery Diagram

Input

↓

result @ = average + 10 ;

↓

Read "result"

↓

ID(result)

↓

Read "@"

↓

Invalid character

↓

Report lexical error

↓

Skip "@"

↓

Read "="

↓

ASSIGN

↓

Read "average"

↓

ID(average)

↓

Read "+"

↓

PLUS

↓

Read "10"

↓

INT_CONST

↓

Read ";

↓

SEMICOLON

Syntax error

A sequence of valid tokens is arranged incorrectly according to the language grammar.

For example:

int = total 250;

contains valid tokens, but their arrangement is syntactically incorrect.

In our program, the first error encountered by the lexical analyzer is the invalid @.

Q4) Design Regular Expressions for Token Recognition

Consider the simplified programming language with the following token categories:

Keywords: int, float, while, return

Identifiers: letter followed by letters or digits

Integer constants

Floating-point constants

Assignment operator: =

Relational operators: <, <=, >, >=, ==

Arithmetic operators: +, -, *, /

Separators: (,), {, }, ;

Input:

```
float total = price * 12.5;
```

```
while (total >= 100) {
```

```
    total = total - 10;
```

```
}
```

```
return total;
```

1. Regular Expressions for Token Categories

A. Keywords

The four keywords can be represented as:

int | float | while | return

Token specification:

KEYWORD $\rightarrow$ (int|float|while|return)

The lexer must additionally ensure that the keyword is not followed by another identifier character. For example, integer should be recognized as an identifier, not as int + eger.

B. Identifiers

The question specifies that an identifier:

- Begins with a letter.
- Is followed by zero or more letters or digits.

Therefore:

$\text{IDENTIFIER} \rightarrow [\text{A-Za-z}][\text{A-Za-z0-9}]^*$

Examples:

total

price

amount1

x

are valid identifiers.

C. Integer Constants

An integer constant consists of one or more digits:

$\text{INTEGER} \rightarrow [0-9]^+$

Examples:

10

100

250

5

D. Floating-Point Constants

A simple floating-point specification is:

$\text{FLOAT} \rightarrow [0-9]^+ \cdot [0-9]^+$

This recognizes a sequence of digits, followed by a decimal point, followed by one or more digits.

Examples:

12.5

100.25

5.0

are floating-point constants.

E. Assignment Operator

$\text{ASSIGN} \rightarrow =$

F. Relational Operators

The relational operators are:

$\text{REL_OP} \rightarrow <= \mid >= \mid == \mid < \mid >$

The multi-character operators should be considered before their shorter prefixes.

For example:

$>=$

must be recognized as one token rather than:

$> =$

The **longest-match rule** handles this.

G. Arithmetic Operators

$\text{ARITH_OP} \rightarrow \backslash+ \mid - \mid \backslash* \mid /$

Here + and * are escaped because they have special meanings in regular expressions.

H. Separators

$\text{SEPARATOR} \rightarrow \backslash(\mid \backslash) \mid \backslash\{ \mid \backslash\} \mid ;$

2. Complete Token Specification

The lexical analyzer can use the following rules:

Token Category	Regular Expression
Keyword	<code>int float while return</code>
Identifier	<code>[A-Za-z][A-Za-z0-9]*</code>
Integer constant	<code>[0-9]+</code>
Floating constant	<code>[0-9]+\.[0-9]+</code>
Assignment	<code>=</code>
Relational operator	<code><= > = < ></code>
Arithmetic operator	<code>\+ \- *</code>
Separator	<code>\(\) \{ \} ;</code>

3. Lexical Analysis of the Input

Line 1

float total = price * 12.5;

The lexer reads from left to right.

float

It matches the keyword pattern:

float → KEYWORD

So:

<KEYWORD,float>

total

It matches:

[A-Za-z][A-Za-z0-9]*

Therefore:

total → IDENTIFIER

=

Matches:

= → ASSIGN

price

Matches the identifier pattern:

price → IDENTIFIER

*

Matches the arithmetic operator pattern:

* → ARITH_OP

More specifically:

* → MULTIPLY

12.5

The lexer sees:

12.5

It matches:

[0-9]+\.[0-9]+

Therefore:

12.5 → FLOAT_CONST

;

Matches the separator pattern:

; → SEMICOLON

Token stream for Line 1

KEYWORD

ID

ASSIGN

ID

MUL

FLOAT_CONST

SEMICOLON

Or with values:

<KEYWORD,float>

<ID,total>

<ASSIGN,=>

<ID,price>

<MUL,*>

<FLOAT_CONST,12.5>

<SEMICOLON,;>

4. Line 2

while (total >= 100) {

while

while → KEYWORD

(

(→ LPAREN

total

total → ID

>=

The lexer sees two possible patterns:

>

>=

Using the **longest-match rule**, it selects:

>= → REL_OP

100

100 → INTEGER_CONST

)

) → RPAREN

{

{ → LBRACE

Token stream

<KEYWORD,while>

<LPAREN,(>

<ID,total>

<REL_OP,>=>

<INTEGER_CONST,100>

<RPAREN,>

<LBRACE,{>

5. Line 3

total = total - 10;

The lexemes are:

Lexeme	Token
total	ID
=	ASSIGN
total	ID

Lexeme	Token
-	MINUS
10	INTEGER_CONST
;	SEMICOLON

Token stream:

<ID,total>

<ASSIGN,=>

<ID,total>

<MINUS,->

<INTEGER_CONST,10>

<SEMICOLON,;>

6. Line 4

}

The closing brace is a separator:

< RBRACE, } >

So:

RBRACE

7. Line 5

return total;

return

It is one of the predefined keywords:

return → KEYWORD

total

total → ID

;

; → SEMICOLON

Token stream:

<KEYWORD,return>

<ID,total>

<SEMICOLON,;>

8. Complete Token Stream

The complete input:

```
float total = price * 12.5;
```

```
while (total >= 100) {
```

```
    total = total - 10;
```

```
}
```

```
return total;
```

produces:

<KEYWORD,float>

<ID,total>

<ASSIGN,=>

<ID,price>

<MUL,*>

<FLOAT_CONST,12.5>

<SEMICOLON,;>

<KEYWORD,while>

<LPAREN,(>

<ID,total>

<REL_OP,>=>

<INTEGER_CONST,100>

<RPAREN,>

<LBRACE,{>

<ID,total>

<ASSIGN,=>

<ID,total>

<MINUS,->

<INTEGER_CONST,10>

<SEMICOLON,;>

<RBRACE,>

<KEYWORD,return>

<ID,total>

<SEMICOLON,;>

In compact form:

KEYWORD ID ASSIGN ID MUL FLOAT_CONST SEMICOLON

KEYWORD LPAREN ID REL_OP INTEGER_CONST RPAREN LBRACE

ID ASSIGN ID MINUS INTEGER_CONST SEMICOLON

RBRACE

KEYWORD ID SEMICOLON

9. How Does the Lexer Distinguish float from an Identifier?

This is an important point.

The identifier pattern is:

[A-Za-z][A-Za-z0-9]*

Technically, float **also matches the identifier pattern**.

Therefore, the lexer first recognizes the sequence of characters:

float

as a word/identifier-like lexeme.

It then checks the **keyword table**.

The keyword table contains:

int

float

while

return

Since float is present in the keyword table:

float → KEYWORD

rather than:

float → ID

But consider:

floating

It is not in the keyword table. Therefore:

floating → ID

Similarly:

float1

is an identifier because the complete lexeme float1 is not the keyword float.

Example

float → KEYWORD

float1 → ID

floating → ID

total → ID

This is normally implemented using **keyword lookup after identifier recognition**.

10. Why Is 12.5 a Floating-Point Constant?

The lexer has two patterns:

Integer

$[0-9]^+$

Floating point

$[0-9]^+\backslash\.[0-9]^+$

For:

12.5

the integer pattern can initially match:

12

But it cannot consume the entire lexeme because of the decimal point.

The floating-point pattern matches the complete sequence:

12.5

Therefore:

12.5 → FLOAT_CONST

and not:

12 → INTEGER_CONST

. → invalid/other token

5 → INTEGER_CONST

The lexer uses the **longest valid match**, so it recognizes 12.5 as one floating-point constant.

11. Importance of Longest-Match Rule

The longest-match rule is particularly important for operators and numbers.

For:

>=

the lexer chooses:

>= → REL_OP

instead of:

> → REL_OP

= → ASSIGN

For:

12.5

it recognizes:

12.5 → FLOAT_CONST

rather than prematurely recognizing:

12 → INTEGER_CONST

Thus, the lexer always tries to form the **longest valid lexeme** from the current input position.

Q5) Complete Compiler Processing

Consider the C statement:

```
int average = total / count;
```

A compiler processes this statement through several phases. Each phase takes the output of the previous phase as its input and produces a new form of representation.

Overall Flow

Source Program



Lexical Analysis



Tokens



Syntax Analysis



Syntax Tree / AST



Semantic Analysis



Annotated AST



Intermediate Code Generation



Intermediate Representation



Code Optimization



Optimized IR



Code Generation



Target Machine Code

1. Lexical Analysis

Input

The lexical analyzer receives the source statement:

int average = total / count;

The lexer reads the characters from left to right and groups them into **lexemes**.

Lexemes and Tokens

Lexeme	Token	Category
int	INT	Keyword
average	ID	Identifier
=	ASSIGN	Operator
total	ID	Identifier
/	DIV	Operator
count	ID	Identifier
;	SEMICOLON	Separator

Therefore, the token stream is:

INT ID ASSIGN ID DIV ID SEMICOLON

More precisely:

<INT>

<ID, average>

<ASSIGN, =>

<ID, total>

<DIV, />

<ID, count>

<SEMICOLON, ;>

The lexical analyzer also removes unnecessary spaces and may record the identifiers in the symbol table.

2. Syntax Analysis

The **syntax analyzer**, or parser, receives the token stream:

INT ID ASSIGN ID DIV ID SEMICOLON

It checks whether the tokens follow the grammar of the programming language.

A simplified grammar could be:

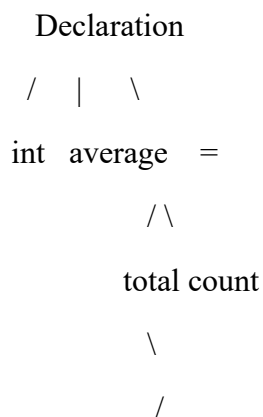
declaration $\rightarrow$ type ID = expression ;

type $\rightarrow$ int

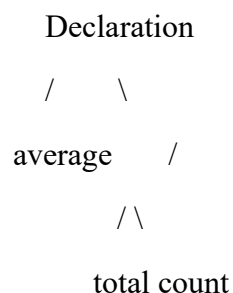
expression $\rightarrow$ expression / ID

| ID

The statement can be represented using an **Abstract Syntax Tree (AST)**.



A clearer AST representation is:



with the declaration type:

average : int

So the syntax structure represents:

average = total / count

where average is declared as an integer.

Output of Syntax Analysis

The output is an AST or syntax structure:

Declaration

```
|
+-- Type: int
|
+-- Identifier: average
|
+-- Expression
    |
    +-- DIV
        / \
        total count
```

3. Semantic Analysis

Syntax analysis only checks whether the statement is **grammatically correct**.

Semantic analysis checks whether the statement **makes sense according to the language rules**.

Input

The semantic analyzer receives the AST:

Declaration

```
|
+-- int
+-- average
+-- DIV
    / \
    total count
```

a. Declaration Checking

The compiler checks whether average is being declared correctly.

`int average;`

is a valid declaration.

It enters information about average into the symbol table.

It also checks whether:

`total`

`count`

have already been declared.

For example:

`int total;`

`int count;`

would make their use valid.

If total or count has not been declared, the compiler reports an error such as:

'total' undeclared

b. Type Checking

Suppose the declarations are:

`int total;`

`int count;`

`int average;`

Then:

`total → int`

`count → int`

`total / count → int`

`average → int`

Therefore:

`int average = total / count;`

is type-compatible.

The semantic analyzer verifies that the result of:

total / count

can be assigned to:

average

c. Scope Checking

The compiler also checks whether total, count, and average are accessible in the current scope.

d. Division Consideration

If total and count are integers, the operation is **integer division** in C.

For example:

$$7 / 2 = 3$$

rather than 3.5.

This is an important semantic/type property of the statement.

4. Symbol Table

The **symbol table** is a data structure maintained by the compiler to store information about identifiers.

For this statement, it could contain:

Name	Type	Kind	Scope
average	int	Variable	Current scope
total	int	Variable	Current scope
count	int	Variable	Current scope

The exact contents depend on the surrounding program.

Role of the Symbol Table

The symbol table is used throughout compilation.

During lexical analysis:

- Identifier information can be recorded.

During syntax analysis:

- Identifier references can be associated with declarations.

During semantic analysis:

- Types are looked up.

- Declarations are verified.
- Scope is checked.
- Type compatibility is checked.

During intermediate-code generation:

- The compiler uses identifier information to generate appropriate operations.

During code generation:

- The compiler determines where variables are stored, such as registers or memory locations.

So the symbol table acts as a common information source throughout the compiler.

5. Intermediate Code Generation

After semantic analysis, the compiler converts the AST into an **Intermediate Representation (IR)**.

One common form is **Three-Address Code (TAC)**.

For:

```
int average = total / count;
```

a suitable representation is:

```
t1 = total / count
```

```
average = t1
```

Here:

- t1 is a temporary variable.
- The division result is stored in t1.
- The result is then assigned to average.

Another possible three-address representation is:

```
t1 = DIV total, count
```

```
average = t1
```

Input

Annotated AST

Output

```
t1 = total / count
```

```
average = t1
```

6. Code Optimization

The middle end may optimize the intermediate representation.

For this simple statement, there may be little optimization possible.

The compiler could potentially represent the operation more directly as:

average = total / count

instead of using a temporary.

The optimization phase attempts to improve:

- Execution speed
- Code size
- Register usage
- Memory access

while preserving the meaning of the program.

7. Code Generation

The **code generator** converts the optimized intermediate representation into target-specific machine instructions.

Assume a simplified register-based processor.

For:

average = total / count

possible target-level instructions could look like:

LOAD R1, total

LOAD R2, count

DIV R1, R2

STORE average, R1

Meaning:

Step 1

LOAD R1, total

Load the value of total into register R1.

Step 2

LOAD R2, count

Load count into register R2.

Step 3

DIV R1, R2

Divide the value in R1 by the value in R2.

The result is stored in R1.

Step 4

STORE average, R1

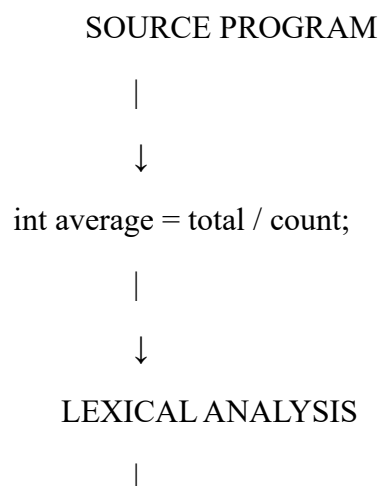
Store the result into average.

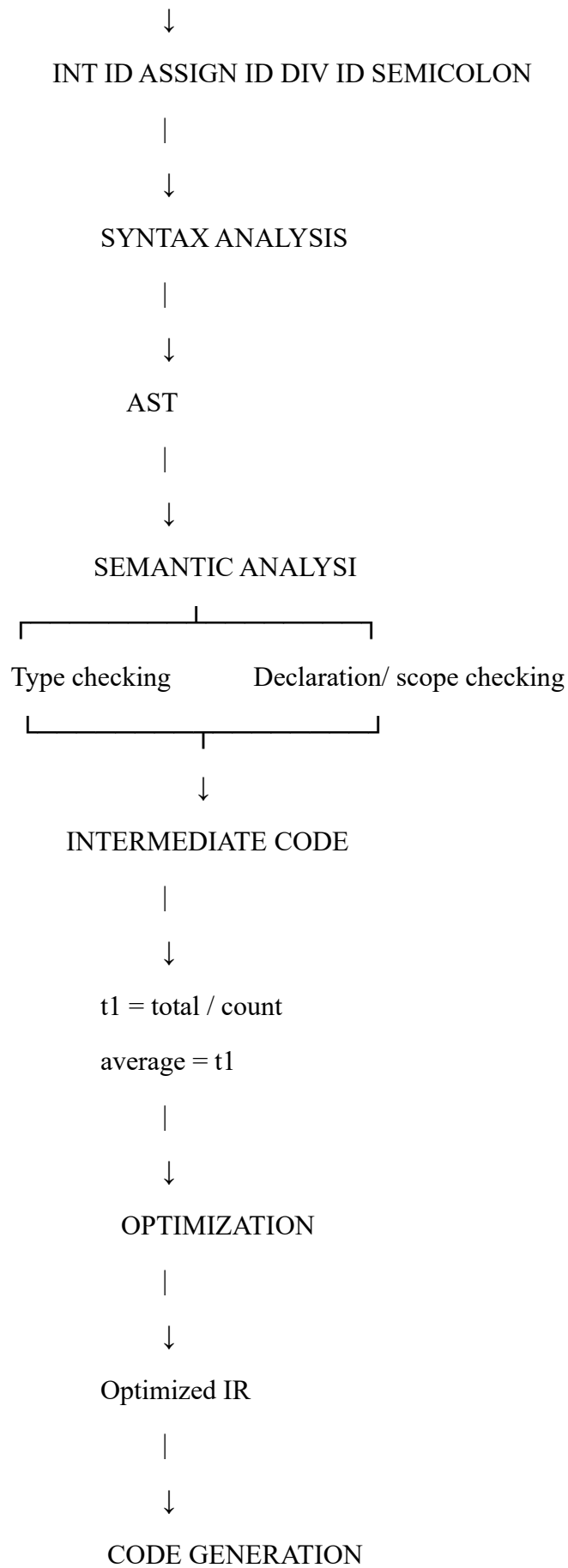
These are illustrative target instructions. Actual instructions depend on the processor architecture and compiler.

8. Complete Phase-by-Phase Table

Compiler Phase	Input	Output
Lexical Analysis	int average = total / count;	Tokens
Syntax Analysis	Token stream	Parse tree / AST
Semantic Analysis	AST + Symbol Table	Type-checked/annotated AST
Intermediate Code Generation	Annotated AST	IR / Three-address code
Optimization	IR	Optimized IR
Code Generation	Optimized IR	Target instructions

9. Complete Processing Flow





|
↓
LOAD R1, total
LOAD R2, count
DIV R1, R2
STORE average, R1

10. Front End, Middle End and Back End

A modern compiler is commonly organized into three broad sections.

A. Front End

The **front end** understands the source program.

It generally includes:

Source Code

↓

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis

For our statement:

int average = total / count;

the front end determines:

- What the words and symbols are.
- Whether the statement follows the grammar.
- What average, total, and count mean.
- Their data types.
- Whether the assignment is semantically valid.

The front end produces an **AST/annotated AST**.

B. Middle End

The **middle end** works with an intermediate representation rather than directly with the source language or machine language.

For example:

$t1 = \text{total} / \text{count}$

$\text{average} = t1$

The middle end performs optimizations such as:

- Removing unnecessary operations.
- Improving register usage.
- Reducing redundant calculations.
- Improving execution efficiency.

Its major advantage is that optimization can be largely **machine-independent**.

C. Back End

The **back end** converts the optimized IR into target-machine code.

For example:

LOAD R1, total

LOAD R2, count

DIV R1, R2

STORE average, R1

The back end deals with machine-specific issues such as:

- Register allocation
- Instruction selection
- Memory locations
- Target-machine instructions

11. Why This Organization Is Useful

Separating the compiler into front end, middle end, and back end makes the compiler easier to design and maintain.

Front end

Source language $\rightarrow$ AST

It handles language-specific details.

Middle end

AST/IR → Optimized IR

It performs mostly machine-independent optimization.

Back end

Optimized IR → Machine code

It handles the target processor.

This separation also allows the same front end and middle-end infrastructure to support multiple target machines. For example, the statement could be translated into x86, ARM, or another architecture by using different back ends.